

TLS HTTPS Wireshark Project Report

Abstract

This project demonstrates the implementation and analysis of secure HTTPS communication using Transport Layer Security (TLS) in a controlled virtualized environment. The project was implemented using two Linux Mint virtual machines configured in VMware Workstation Pro. A complete Public Key Infrastructure (PKI) was developed using OpenSSL by creating a Root Certificate Authority (CA), generating server and client certificates, and establishing authenticated TLS communication.

The project also involved capturing TLS network traffic using Wireshark, analyzing the TLS handshake process, and decrypting encrypted HTTPS traffic using exported session keys through SSLKEYLOGFILE. The implementation demonstrates practical understanding of TLS 1.3, certificate-based authentication, cryptographic protocols, HTTPS communication, and network traffic analysis.

Student Information

Field	Information
Student Name	Yassine Hesham Shehata
Registration Number	231002215
Course	CCY3201
Project Title	TLS HTTPS Implementation and Analysis

Table of Contents

1. Introduction
2. Environment Setup
3. Tools and Technologies
4. Root Certificate Authority Creation
5. Server Certificate Generation
6. Client Certificate Generation
7. HTTPS TLS Server Implementation
8. TLS Client Connection
9. HTTPS Communication
10. Wireshark TLS Analysis
11. TLS Traffic Decryption
12. Security Discussion
13. Conclusion

1. Introduction

1.1 Background

Transport Layer Security (TLS) is a cryptographic protocol designed to provide confidentiality, integrity, and authentication for communications over computer networks. TLS is widely used in HTTPS websites to secure communication between clients and servers and to protect sensitive information from interception or modification.

TLS uses a combination of asymmetric cryptography, symmetric encryption, digital certificates, and cryptographic hashing to establish secure communication channels.

1.2 Project Objectives

The objectives of this project were:

- To build a complete TLS environment using Linux virtual machines
 - To generate and manage TLS certificates using OpenSSL
 - To implement a secure HTTPS TLS server
 - To establish encrypted HTTPS communication
 - To analyze TLS packets using Wireshark
 - To decrypt TLS traffic using exported session secrets
 - To understand the TLS handshake process and modern cryptographic protocols
-

2. Environment Setup

2.1 Virtualization Platform

The project environment was implemented using VMware Workstation Pro.

2.2 Virtual Machines

Two Linux Mint virtual machines were configured:

Virtual Machine	Purpose	IP Address
Server VM	TLS HTTPS Server	192.168.50.10
Client VM	TLS Client	192.168.50.20

2.3 Network Configuration

Two VMware network adapters were configured:

Adapter Type	Purpose
Host-only Adapter	Communication between VMs
NAT Adapter	Internet connectivity

This configuration allowed secure isolated communication between the client and server virtual machines while maintaining internet access for package installation.

3. Tools and Technologies

The following tools and technologies were used during implementation:

Tool	Purpose
OpenSSL	Certificate generation and TLS communication
Wireshark	Network packet capture and analysis
Firefox	HTTPS browser testing
VMware Workstation Pro	Virtualization platform
Linux Mint	Operating system environment

Required packages were installed using:

```
sudo apt update
sudo apt install openssl wireshark net-tools curl gcc make -y
```

4. Root Certificate Authority Creation

4.1 Root CA Private Key

The Root Certificate Authority private key was generated using OpenSSL:

```
openssl genrsa -out rootCA.key 4096
```

The generated private key was secured using Linux file permissions:

```
chmod 600 rootCA.key
```

This ensured that only the file owner could access the Root CA private key.

4.2 Root CA Certificate

The self-signed Root CA certificate was generated using:

```
openssl req -x509 -new -nodes -key rootCA.key -sha256 -days 3650 -out rootCA.crt
```

The Root CA certificate was later used to sign both the server and client certificates.

5. Server Certificate Generation

5.1 Server Private Key

The server private key was generated using:

```
openssl genrsa -out server.key 2048
```

The private key was protected using:

```
chmod 600 server.key
```

5.2 Server Certificate Signing Request

A Certificate Signing Request (CSR) was created for the server:

```
openssl req -new -key server.key -out server.csr
```

The Common Name (CN) used during CSR creation was the server IP address:

```
192.168.50.10
```

5.3 Server Certificate Signing

The server certificate was signed using the Root CA:

```
openssl x509 -req -in server.csr -CA rootCA.crt -CAkey rootCA.key -  
CAcreateserial -out server.crt -days 365 -sha256
```

The generated server certificate enabled authenticated TLS communication.

6. Client Certificate Generation

6.1 Client Private Key

The client private key was generated using:

```
openssl genrsa -out client.key 2048
```

6.2 Client CSR Generation

A Certificate Signing Request was generated for the client:

```
openssl req -new -key client.key -out client.csr
```

The client IP address was used as the Common Name:

```
192.168.50.20
```

6.3 Client Certificate Signing

The client certificate was signed using the Root CA:

```
openssl x509 -req -in client.csr -CA rootCA.crt -CAkey rootCA.key -  
CAcreateserial -out client.crt -days 365 -sha256
```

7. HTTPS TLS Server Implementation

7.1 HTML Webpage

A simple HTML webpage was created and hosted on the TLS server:

```
<html>  
<head>  
<title>CCY3201 TLS Project</title>  
</head>  
<body>  
<h1>Welcome to My TLS Server</h1>  
<p>HTTPS communication secured using TLS.</p>  
</body>  
</html>
```

7.2 TLS HTTPS Server

The HTTPS TLS server was implemented using OpenSSL:

```
openssl s_server -accept 4433 -cert server.crt -key server.key
```

The server successfully listened for incoming TLS connections on TCP port 4433.

8. TLS Client Connection

The TLS client connected successfully to the HTTPS server using:

```
openssl s_client -connect 192.168.50.10:4433
```

The negotiated TLS version was:

```
TLS 1.3
```

The negotiated cipher suite was:

```
TLS_AES_256_GCM_SHA384
```

The TLS handshake completed successfully and established encrypted communication between the client and server.

9. HTTPS Communication

After successful TLS negotiation, an encrypted HTTP GET request was sent from the client to the server:

```
GET /index.html HTTP/1.1  
Host: 192.168.50.10
```

The server responded successfully with the HTML webpage through the encrypted TLS channel.

This confirmed successful HTTPS communication using TLS encryption.

10. Wireshark TLS Analysis

Wireshark was used to capture and analyze TLS traffic exchanged between the client and server.

The following capture filter was used:

```
tcp port 4433
```

10.1 TLS Client Hello

The Client Hello packet initiated the TLS handshake and advertised:

- Supported TLS versions
- Supported cipher suites
- Cryptographic extensions
- Key exchange parameters

10.2 TLS Server Hello

The Server Hello packet selected:

- TLS version
- Cipher suite
- Cryptographic parameters

The server selected TLS 1.3 and a secure AES-GCM cipher suite.

10.3 Certificate Exchange

During the TLS handshake, the server transmitted its certificate to authenticate its identity.

The client validated the certificate chain using the Root CA certificate.

10.4 Encrypted Application Data

After handshake completion, all communication became encrypted TLS Application Data.

This ensured confidentiality and integrity of HTTP traffic.

11. TLS Traffic Decryption

11.1 SSLKEYLOGFILE

TLS session secrets were exported using:


```
export SSLKEYLOGFILE=~/.tlskeys/keylogfile.txt
```

Firefox was launched from the same terminal to ensure session secrets were written to the key log file.

11.2 Wireshark TLS Decryption

The exported key log file was imported into Wireshark using:

```
Edit → Preferences → Protocols → TLS
```

The following key log file path was configured:

```
/home/yassine_client/.tlskeys/keylogfile.txt
```

Wireshark successfully decrypted encrypted TLS packets and revealed plaintext HTTP traffic.

The decrypted packets included:

- HTTP GET request
- HTTP response
- HTML webpage content

12. Security Discussion

12.1 TLS 1.3

TLS 1.3 provides improved security and performance compared to previous TLS versions.

Advantages include:

- Reduced handshake latency
- Forward secrecy
- Stronger cryptographic algorithms
- Removal of insecure legacy ciphers

12.2 Cipher Suite

The negotiated cipher suite was:

TLS_AES_256_GCM_SHA384

This cipher suite provides:

Component	Purpose
AES-256	Symmetric encryption
GCM	Authenticated encryption
SHA384	Integrity and hashing

12.3 X25519 Key Exchange

X25519 was used for ephemeral key exchange and forward secrecy.

This ensures that even if long-term keys are compromised in the future, previously captured TLS sessions cannot be decrypted.

12.4 RSA-PSS Signatures

RSA-PSS signatures were used for certificate authentication and digital signatures.

RSA-PSS provides stronger security than older RSA signature schemes.

12.5 Browser Security Warning

Firefox displayed a security warning because the Root CA used in the project was self-signed and not trusted by the operating system certificate store.

This behavior is expected in lab environments and does not indicate a TLS failure.

13. Conclusion

This project successfully demonstrated the implementation and analysis of TLS-secured HTTPS communication using OpenSSL and Wireshark.

A complete TLS environment was implemented including:

- Root Certificate Authority creation
- Server and client certificate generation
- HTTPS TLS server implementation
- TLS handshake analysis
- TLS traffic decryption
- Wireshark packet inspection

The project provided practical understanding of Public Key Infrastructure (PKI), HTTPS encryption, certificate-based authentication, TLS handshake mechanisms, and secure network communication.

The implementation successfully achieved all project objectives and demonstrated modern secure communication using TLS 1.3.

14. Deliverables

The following files were included in the final submission:

File	Description
rootCA.crt	Root Certificate Authority certificate
server.crt	TLS server certificate
client.crt	TLS client certificate
tls_client_server_urIDs.pcapng	Encrypted TLS capture
tls_decrypted_client_server_urIDs.pcap	Decrypted TLS capture
index.html	Hosted HTTPS webpage
Project screenshots	TLS implementation screenshots
Project report	Final documentation

15. GitHub Repository

The project repository includes:

- certificates
- packet captures
- screenshots
- report

- HTML webpage
- README documentation

Repository structure:

```
TLS-HTTPS-Wireshark-Project/  
|  
├─ certificates/  
├─ captures/  
├─ screenshots/  
├─ website/  
├─ report/  
└─ README.md
```

End of Report